

Structure-Preserving Doubling for the Algebraic Riccati Equations

Amp

A plant with a growing mode forces a controller out of a matrix equation that is quadratic in its unknown, so it has several answers and only one of them is the controller. This note asks six questions about getting that answer without an eigensolver. Which of the several solutions does the gain come from, and what singles it out? Why do the two obvious iterations each buy only a fixed number of digits per step? What identity lets the 2^{k+1} -term partial sum of a linear matrix equation be assembled from one 2^k -term partial sum? Why does the update that follows, written only in the data, leave the solution where it is while squaring the contraction? How does a continuous-time plant get into a discrete-time doubling? And what goes wrong when the shift is chosen badly or a multiplier reaches the unit circle? Each question is set up against one two-state LQR problem and one figure, table, or sidecar sweep, so an answer can be checked against the problem rather than against this page.

October 01, 2026

Contents

1. One quadratic matrix equation, several answers	3
2. Why the obvious iterations crawl	4
3. Squaring a partial sum	5
4. Why the update never mentions the unknown	6
5. Getting a continuous plant into a discrete doubling	7
6. Where the guarantee stops	8
7. Sources and further reading	9

1. One quadratic matrix equation, several answers

Question: The gain of the running plant is read off a matrix equation that is quadratic in its unknown, so it has more than one symmetric solution, and one mode of the plant grows. Which of those solutions does the controller use, and what property of the closed loop singles it out?

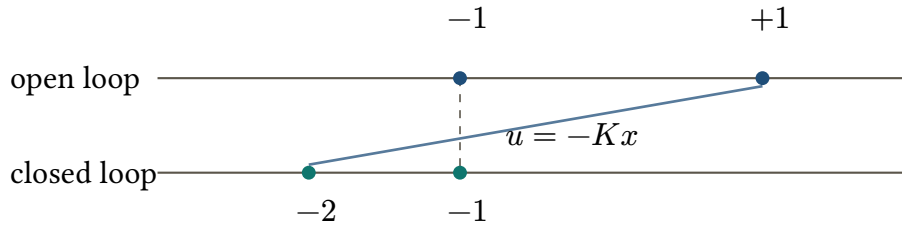


Figure 1: The plant of Equation 1 before and after the feedback. Inspect which pole the feedback moves, which one it leaves where it was, and how far the moved one travels.

The running example. A two-state plant $\dot{x} = Ax + Bu$ with

$$A = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad Q = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad R = 1 \quad (1)$$

and cost $J = \int_0^\infty (x^T Q x + u^2) dt$. The first state decays on its own; the second grows like e^t , so feedback is what keeps the state bounded.

The two equations. Write $G = BR^{-1}B^T = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. The optimal control is a static feedback $u = -Kx$, and the classical LQR result – used here, not re-derived – reads the gain K off a symmetric solution X of

$$K = B^T X, \quad A^T X + XA - XGX + Q = 0, \quad (2)$$

the continuous-time algebraic Riccati equation (CARE). Its discrete-time sibling is the equation an algorithm that doubles actually runs on,

$$X = Q + A^T X A - A^T X B (R + B^T X B)^{-1} B^T X A = Q + A^T X (I + GX)^{-1} A, \quad (3)$$

the discrete-time algebraic Riccati equation (DARE).

Facts to reason with.

- Both equations are quadratic in X , so neither is a linear system, and the CARE on Equation 1 has several symmetric solutions.
- $A - GX$ is the closed loop of the feedback. A matrix is called stable here when every one of its eigenvalues lies in the open left half-plane, and in the discrete case inside the open unit disk.
- The textbook route to a symmetric solution builds the $2n \times 2n$ Hamiltonian matrix $\begin{pmatrix} A & -G \\ -Q & -A^T \end{pmatrix}$ and works on it with an ordered Schur decomposition.
- This note asks whether the controller's solution can be had from $n \times n$ linear algebra alone – no eigenvalues, no Schur form – and how many such solves it costs.

2. Why the obvious iterations crawl

Question: The Riccati equation is a fixed-point equation in disguise, so there are two obvious ways to iterate it: integrate the Riccati flow that solves the finite-horizon problem, or iterate the recursion Equation 4 from $X_0 = 0$. Both converge, and both are built from steps that are individually exact and cheap. Predict what fixes the number of digits each step gains, and what a single step would have to do to the error to gain the digits of many plain steps at once.

TARGET ERROR	RICCATI FLOW, RK4 $dt = 0.02$	PLAIN RECURSION
10^{-4}	249	5
10^{-8}	479	9
10^{-12}	710	13

Table 1: Steps each route needs to drive the relative error of its iterate below 10^{-4} , 10^{-8} and 10^{-12} on the running example, from *scripts/verify_sda_are.py*. Inspect how each column grows with the target, and compare the two step sizes.

Route one, the flow. The finite-horizon Riccati differential equation $\dot{Y} = Q + A^T Y + Y A - Y G Y$, started from $Y(0) = 0$, rises to X as the horizon grows. Near the limit the error obeys $\dot{\Delta} \approx (A - G X)^T \Delta + \Delta (A - G X)$.

Route two, the recursion. Iterating Equation 3 from $X_0 = 0$ gives

$$X_{j+1} = Q + A^T X_j (I + G X_j)^{-1} A, \quad (4)$$

whose error contracts by σ^2 per step, where $\sigma = \rho((I + G X)^{-1} A) < 1$ and ρ is the spectral radius. The table's recursion column is this iteration on the transformed data of section 5, and the flow column is the differential equation on the data of Equation 1.

Facts to reason with.

- The closed-loop poles of the running example are -1 and -2 ; the transformed problem of section 5 has multiplier $\sigma = \frac{1}{3}$.
- One step of either route costs $O(n^3)$ work and advances the iterate index by one.
- Both routes stay inside $n \times n$ linear algebra; the cost being examined here is the number of steps, not the work in a step.

3. Squaring a partial sum

Question: Strip the quadratic term out and the equation becomes the linear Stein equation $X = A^T X A + Q$ with $\rho(A) < 1$, whose solution is the sum $\sum_{i \geq 0} (A^T)^i Q A^i$. Iterating $X_{j+1} = A^T X_j A + Q$ from zero produces the partial sums of that series. What identity lets the 2^{k+1} -term partial sum be assembled from one 2^k -term partial sum, and what recursion on the data does that identity give?

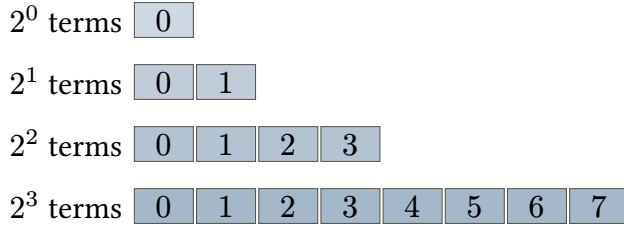


Figure 2: The partial sums of the Stein series, one row per k : row k holds the 2^k terms $(A^T)^i Q A^i$ with $i = 0, \dots, 2^k - 1$, in order. Inspect how the row for $k + 1$ is built from the row for k .

Setup. For the Stein equation the iterate from $X_0 = 0$ is exactly

$$X_j = \sum_{i=0}^{j-1} (A^T)^i Q A^i, \quad \text{so} \quad X - X_j = (A^T)^j X A^j. \quad (5)$$

The error is one constant factor per step and nothing more.

Facts to reason with.

- X_{2^k} is a partial sum with 2^k terms, and every term is built from A and Q .
- The jump must be exact: whatever identity assembles the larger sum introduces no approximation.
- On a 2×2 Stein equation with $\rho(A) = 0.688$ the plain partial sums gain about 0.325 digits per step.

4. Why the update never mentions the unknown

Question: Put the quadratic term back. The doubling can be written entirely in the data (A_k, G_k, Q_k) , with no reference to the unknown X , as Equation 6. Why does the same X solve the Riccati equation at every k , and why is the contraction of the k -th problem the 2^k -th power of the first one's?

k	$\ A_k\ _F$	σ_k	RESIDUAL OF X
0	1.01×10^0	3.33×10^{-1}	1.92×10^{-16}
1	3.86×10^{-1}	1.11×10^{-1}	4.84×10^{-16}
2	4.36×10^{-2}	1.24×10^{-2}	4.84×10^{-16}
3	5.39×10^{-4}	1.52×10^{-4}	5.00×10^{-16}
4	8.21×10^{-8}	2.32×10^{-8}	4.81×10^{-16}
5	1.91×10^{-15}	5.40×10^{-16}	4.81×10^{-16}

Table 2: The data of the k -th problem of Equation 6 and the residual of the original solution X in that problem's Riccati equation, from *scripts/verify_sda_are.py*. Inspect which columns move with k and which one does not.

The update. From $A_0 = A, G_0 = G, Q_0 = Q$,

$$\begin{aligned}
 A_{k+1} &= A_k(I + G_k Q_k)^{-1} A_k, \\
 G_{k+1} &= G_k + A_k(I + G_k Q_k)^{-1} G_k A_k^T, \\
 Q_{k+1} &= Q_k + A_k^T Q_k(I + G_k Q_k)^{-1} A_k.
 \end{aligned} \tag{6}$$

Facts to reason with.

- The multiplier of a DARE with data (A, G, Q) is $K = (I + GX)^{-1}A$, the matrix that carries the state through one closed-loop step.
- A Riccati equation is also a statement about a subspace. If X solves Equation 3, then

$$\begin{pmatrix} A & 0 \\ -Q & I \end{pmatrix} \begin{pmatrix} I \\ X \end{pmatrix} = \begin{pmatrix} I & G \\ 0 & A^T \end{pmatrix} \begin{pmatrix} I \\ X \end{pmatrix} K, \tag{7}$$

so the block column $\begin{pmatrix} I \\ X \end{pmatrix}$ is a deflating subspace of the pencil formed by the two blocks in Equation 7, and K is the operator that pencil induces on it.

- The only inverse in Equation 6 is $I + G_k Q_k$, and G and Q are symmetric.

5. Getting a continuous plant into a discrete doubling

Question: The update Equation 6 doubles a discrete-time Riccati equation, but the running plant is continuous. A shift $\tau > 0$ is applied to the plant's data, and the transformed problem of the running example has multiplier $\sigma = \frac{1}{3}$. What must that shift preserve for the doubling's guarantee to carry over, and why does the run in the table reach X in a handful of steps?

k	DIGITS OF Q_k AGREEING WITH X	RESIDUAL OF Q_k
0	0.70	1.92×10^{-16}
1	1.60	4.84×10^{-16}
2	3.50	4.84×10^{-16}
3	7.32	5.00×10^{-16}
4	14.91	4.81×10^{-16}
5	15.88	4.81×10^{-16}

Table 3: The update Equation 6 run on the transformed data of Equation 9, one row per step, from *scripts/verify_sda_are.py*. Inspect how the agreement with X and the residual move as k advances.

The bridge. Choose a shift $\tau > 0$ for which $A_\tau = A - \tau I$ is invertible and set

$$\begin{aligned} K_\tau &= A_\tau^T + Q A_\tau^{-1} G, & A_0 &= I + 2\tau(K_\tau^T)^{-1}, \\ G_0 &= 2\tau A_\tau^{-1} G K_\tau^{-1}, & Q_0 &= 2\tau K_\tau^{-1} Q A_\tau^{-1}. \end{aligned} \quad (8)$$

The shift replaces each eigenvalue λ by $\lambda_\tau = \frac{\lambda + \tau}{\lambda - \tau}$, and Equation 8 is the corresponding replacement of the CARE's data.

The running example. At $\tau = 2$ the transformed data are exactly

$$A_0 = \begin{pmatrix} -\frac{2}{7} & \frac{6}{7} \\ \frac{1}{7} & -\frac{3}{7} \end{pmatrix}, \quad G_0 = \begin{pmatrix} \frac{1}{7} & \frac{3}{7} \\ \frac{3}{7} & \frac{9}{7} \end{pmatrix}, \quad Q_0 = \begin{pmatrix} \frac{3}{7} & -\frac{2}{7} \\ -\frac{2}{7} & \frac{20}{7} \end{pmatrix}, \quad (9)$$

and the transformed problem has multiplier $\sigma = \frac{1}{3}$. One step of Equation 6 is one $n \times n$ inverse plus a handful of products, and the run needs no eigensolver and no stabilizing initial guess.

6. Where the guarantee stops

Question: The shift τ in Equation 8 is free, and the table records the multiplier it produces. Why is there a best τ , and what goes wrong as τ moves away from it in either direction? Then change the hypothesis $\sigma < 1$: what does the doubling become when a closed-loop multiplier reaches modulus one?

Open [sda-algebraic-riccati-demo.html](#) beside this note to move τ and watch the two closed-loop multipliers as they change.

τ	0.25	0.5	1	$\sqrt{2}$	2	4	8	16	32
σ	0.778	0.600	–	0.172	0.333	0.600	0.778	0.882	0.939

Table 4: The multiplier $\sigma(\tau)$ of the transformed problem as the shift moves, from *scripts/verify_sda_are.py*. Inspect where the bridge stops existing and how σ behaves on either side of it.

The knob. Same plant, same algorithm, different τ . The transformed closed-loop multipliers $\left| \frac{\lambda_i + \tau}{\lambda_i - \tau} \right|$ depend on τ , and the table records the resulting $\sigma(\tau)$.

Facts to reason with.

- At $\tau = 1$ the bridge does not merely perform badly: $A - \tau I$ is singular because A has an eigenvalue at $+1$, so Equation 8 has no inverse to use.
- A multiplier of modulus exactly 1 is possible, and its smallest instance is a 2×2 Jordan block $J = \begin{pmatrix} \lambda & 1 \\ 0 & \lambda \end{pmatrix}$ with $|\lambda| = 1$.
- Every step of Equation 6 must invert $I + G_k Q_k$, and that matrix can become singular or badly conditioned along the way.
- The signs of Q and G are hypotheses too: with an indefinite Q there are problems with no stabilizing solution at all.

7. Sources and further reading

No external sources have been consulted yet, and this note makes no claim about prior work, attribution, or the history of any method. Every quantity stated above is either an arithmetic consequence of Equation 1 or a float64 run of Equation 6, reproduced by *scripts/verify_sda_are.py* the sidecar *sda-algebraic-riccati-demo.html* sweeps the shift. The expected answers to the six questions, and the reading that would ground them, are kept in *summary.md* rather than in the document.